

Reviewer Pack — Minimal Rank of Free Probability Entanglement over Read-Twic...

Entry df5829c4c2c8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 16:25:44 UTC

Minimal Rank of Free Probability Entanglement over Read-Twice BP Width

Entry ID: df5829c4c2c8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 16:25:44 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: Read-Twice BP Width

Statement:

{‘text’: ‘For any read-twice branching program P , the minimal rank of its associated free probability entanglement tensor is upper bounded by $O(\log^2 \text{size}(P))$ and lower bounded by $\Omega(\text{size}(P))$.’, ‘counterexample’: ‘Construct a read-twice BP with an entanglement tensor whose minimal rank is less than $\log^2 \text{size}(P)$.’}

Rationale (proposer’s reasoning):

{‘text’: ‘Free probability theory provides a framework for studying quantum information, and its entanglement tensors can capture the complexity of quantum processes. The relationship between these tensors and branching program width could potentially reveal new insights into computational complexity.’}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8328ca727cbab326

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, after generating 30 random read-twice BP instances of size $n \leq 40$, at least 80% of the associated free probability entanglement tensors have a minimal rank that lies within the bounds of $O(\log^2 \text{size}(P))$ and $\Omega(\text{size}(P))$, with no instance having a minimal rank less than $\log^2 \text{size}(P)$. The conjecture is falsified if any seed produces an entanglement tensor with a minimal rank below $\log^2 \text{size}(P)$ or if the support fraction is less than 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability theory" AND "read-twice BP width" - "minimal rank" free probability entanglement AND read-twice BP width - "entanglement tensor minimal rank" AND complexity theory read-twice BP

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log2(x):
        if x <= 0:
            return float('-inf')
        return math.log2(x)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def binomial_coefficient(n, k):
        if k > n:
            return 0
        return factorial(n) // (factorial(k) * factorial(n - k))

    def generate_read_twice_bp(size: int):
        # Simplified representation of a read-twice branching program
        bp = []
        for i in range(size):
            bp.append(random.choice([0, 1]))
        return bp
```

```

def compute_entanglement_tensor(bp):
    size = len(bp)
    tensor = [[0] * size for _ in range(size)]
    for i in range(size):
        for j in range(size):
            if bp[i] == bp[j]:
                tensor[i][j] = 1
    return tensor

def min_rank(tensor):
    rows, cols = len(tensor), len(tensor[0])
    rank = 0
    for _ in range(min(rows, cols)):
        pivot_row = -1
        for r in range(rank, rows):
            if tensor[r][rank] != 0:
                pivot_row = r
                break
        if pivot_row == -1:
            break
        for c in range(cols):
            tensor[pivot_row][c], tensor[rank][c] = tensor[rank][c], tensor[pivot_row][c]
        for r in range(rows):
            if r != rank:
                factor = tensor[r][rank] / tensor[rank][rank]
                for c in range(cols):
                    tensor[r][c] -= factor * tensor[rank][c]
        rank += 1
    return rank

size = random.randint(5, 40)
bp = generate_read_twice_bp(size)
entanglement_tensor = compute_entanglement_tensor(bp)
min_rank_value = min_rank(entanglement_tensor)

lower_bound = size
upper_bound = log2(size) ** 2

return {
    "metric_name": "minimal_rank",
    "metric_value": min_rank_value,
    "instances_tested": 1,
    "conjecture_holds": lower_bound <= min_rank_value <= upper_bound,
    "counterexample": "" if lower_bound <= min_rank_value <= upper_bound else f"rank={min_rank_value}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

total_metric_value = sum(r["metric_value"] for r in results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={total_metric_value / len(results):.2f} std=0.00 support_fr")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={total_metric_value / len(results):.2f} std=0.00 support_fr")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"rank out of bounds\" first_failing_seed={first_")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

False, 'counterexample': 'rank=2, expected=(7, 7.8812416584010565)'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5370477c.py", line 110, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5370477c.py", line 110, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for at least one seed, the minimal rank of the associated free probability entanglement tensor is below $\log^2 \text{size}(P)$, whic | next: Investigate further to understand why the test crashed and whether it was a fluke or indicative of a deeper issue with the conjecture. If the crash is due to a programming error, fix the code and rerun the tests.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10360
2	preregistration	ollama_remote	glm4:latest	0	0	6430
3	novelty	ollama_remote	glm4:latest	0	0	4630
4	novelty	ollama_remote	glm4:latest	0	0	5407
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40489
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9397
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6803
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11561
9	judge	ollama_remote	glm4:latest	0	0	11112

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106189 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/df5829c4c2c8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/df5829c4c2c8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/df5829c4c2c8.tar.gz` (if generated)